eBPF Scheduler Proposal: A Multi-dimensional Multilevel Feedback Queue Scheduler

Jeffrey Jiang
Stony Brook University

1 Motivation

Kernel schedulers are responsible for determining how to best utilize the CPU while remaining performant and flexible under different workloads. The scheduler must be considerate of the different characteristics of the processes being scheduled. Two such important properties that a scheduler can take into account for its decisions are its latency requirements and its ideal time slice length. However, existing schedulers such as FreeBSD's ULE scheduler often conflate these two properties together. I argue that schedulers should instead treat these two properties as orthogonal to each other.

As an example of how schedulers conflate these two properties, consider the ULE scheduler. The ULE scheduler groups processes into two categories, interactive processes and batch processes, based on an *interactivity* metric which is loosely the ratio between the amount of time a thread spends running to the amount of time a thread spends voluntarily sleeping. ULE always schedule interactive processes before batch processes because interactive processes are considered latency sensitive [1]. Thus, ULE associates latency sensitivity with how the proportion that a process spends running. While, this dichotomy works well, it ignores processes which do not neatly fit into this dichotomy. For example, a process which spends a lot of time on computations but needs to respond quickly to an event would be penalized by ULE because it would be considered a batch process. The scheduler I am proposing will solve this issue by treating latency requirements and its ideal time slice length orthogonally.

2 Proposed Scheduler

My proposed scheduler is based on the multilevel feedback queue (MLFQ) scheduling [2] like FreeBSD's ULE except modifications were made to treat the latency requirement and the ideal time slice length orthogonally.

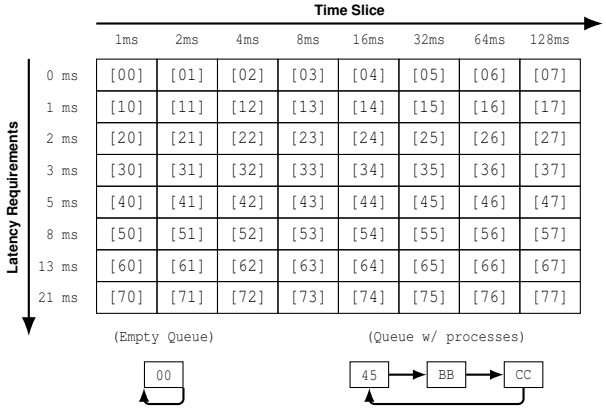


Figure 1: The scheduler is arranged in a two-dimensional grid. Each cell of the grid is a circular feedback run queue. The queue at row 0 and column 0 is empty while the queue at row 4 and column 5 contains two processes: `AA` and `BB`. Times are subject to change.

2.1 Data Structure

My scheduler will use a two-dimensional grid of feedback run queues. Figure 1 shows the structure of the scheduler I am proposing. The grid is 8×8, and each cell is a run queue, represented by a circular linked list. The x-axis of the grid represent the time slice length assigned processes within queues of that column, and the y-axis represents the latency requirement of the process. Together, if a process is in the run queue at row i and column j, then it has the latency requirement assigned to row i and the quantum at column j. The 8×8 grid was chosen, so there is a total of 64 queues. This allows a summary of the grid represented as a bitset in a 64-bit integer which allows us to use bitwise operations to perform operations such as finding the first queue with a process on it.

2.2 Scheduling Policy

Similar to MLFQ, the scheduler will adaptively move processes between different queues with the goal to find the ideal

queue which best suits the process' behavior. Because of the grid structure, this involves determining the time slice length and the latency requirement of the process dynamically.

- **Time Slice Length:** Move the process up a tier if a process completes its allotted time slice. Move down a tier if a process yields before half of its allotted time slice.
- **Latency Requirement:** Latency requirement is determined via user-provided hint as well as inspecting the wait queue that the process awoke from. For example, a process from a futex wait queue has a higher latency requirement than a process waking up due to a network packet.

When a process is ready to be scheduled, the scheduler computes the correct run queue in the grid to place this process within. At this point in time, the scheduler will preempt the currently running process if the remaining time slice exceeds the newly awoken process' latency requirement. Otherwise, the process is simply placed at the tail of the run queue.

When a process completes its time slice, the process is preempted and placed back into the run queue with a latency requirement penalty to allow another process to be scheduled. To select the next process to run, the scheduler will find the first non-empty run queue that first has the lowest latency requirement, then the lowest time slice. From this run queue, the scheduler will schedule the process at the head of the run queue. Currently, this design permits starvation. This, however, can be fixed by performing a deficit round robin instead when selecting the run queue [3].

3 Evaluation Plan

Evaluation will be performed using a mix of synthetic and real-world benchmarks on a bare-metal laptop with an Intel Core i7-6700HQ running Linux Mint. This CPU has a total of 4 cores with 2 threads per core from hyper-threading.

I will be comparing my scheduler against the EEVDF scheduler and potentially the ULE scheduler implemented in eBPF if time permits. First, I will determine the cost of the operations involved in my scheduler and compare it with the other schedulers. I will consider the average and tail latency here. The goal is to demonstrate that my scheduler algorithm is feasible for real-world use.

Next, I will evaluate how well my scheduler performs compared to the baseline. I will use hackbench which is a traditional benchmark and stress test for the Linux schedulers and Schbench which provides latency distribution statistics. I will also construct synthetic benchmarks which represent processes which are compute-heavy and also latency-sensitive and processes which sleep often but are latency-insensitive. Based on my hypothesis, I would expect the baseline schedulers to schedule theses processes poorly, and I would like to examine if my scheduler performs better. For this, I will examine how these processes interfere with batch and interactive processes.

4 Expected Outcomes

I would expect that my scheduler performs performs better than the baseline schedulers on some benchmarks, but not all. Existing schedulers have been highly optimized and the eBPF overhead for my scheduler may be significant due to the expected low cost of scheduling. However, as laid out currently, my scheduling algorithm should have an algorithmic complexity of $O(1)$ compared to the $O(\log n)$ complexity of EEVDF. I do expect that my scheduler outperforms EEVDF and ULE when it comes to the non-traditional processes described in Section 3.

5 Expected Challenges

I expect implementation of the scheduler to be simple due to its simple data structure and algorithms involved. However, the main challenge that I believe I will run into is tuning the parameters of the scheduler. The current design allows a lot of potential adjustments such as determining the how the latency requirement and time slice length scales between tiers. I believe this tuning will be the most crucial in obtaining optimal performance. This will require a further experimentation after the implementation which may be the most time-consuming part.

References

[1] Justinien Bouron, Sebastien Chevalley, Baptiste Lepers, Willy Zwaenepoel, Redha Gouicem, Julia Lawall, Gilles Muller, and Julien Sopena. The battle of the schedulers: FreeBSD ULE vs. linux CFS. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*, pages 85–96, Boston, MA, July 2018. USENIX Association.

[2] Fernando J. Corbató, Marjorie Merwin-Daggett, and Robert C. Daley. An experimental time-sharing system. In *Proceedings of the May 1-3, 1962, Spring Joint Computer Conference*, AIEE-IRE '62 (Spring), page 335–344, New York, NY, USA, 1962. Association for Computing Machinery.

[3] M. Shreedhar and George Varghese. Efficient fair queueing using deficit round robin. In *Proceedings of the Conference on Applications, Technologies, Architectures, and Protocols for Computer Communication*, SIGCOMM '95, page 231–242, New York, NY, USA, 1995. Association for Computing Machinery.